



Strapi সম্পূর্ণ গাইড

শূন্য থেকে Production পর্যন্ত — সহজ ভাষায়, ধাপে ধাপে

Content-Type Builder • CRUD • Roles & Permissions • Deployment

সূচিপত্র

১. Strapi কী এবং কেন ব্যবহার করব

Strapi হলো একটি Headless CMS (Content Management System)। সহজ ভাষায় বুঝতে হলে দুটো বিষয় খেয়াল করুন:

- "Headless" মানে এর নিজস্ব কোনো ফ্রন্টএন্ড (যেটা ব্যবহারকারী সরাসরি দেখে) নেই — শুধু একটা Backend আছে যেটা ডাটা সংরক্ষণ করে এবং API-এর মাধ্যমে সরবরাহ করে।
- আপনি Strapi দিয়ে ডাটা ম্যানেজ করবেন (প্রোডাক্ট, ইউজার, ব্লগ পোস্ট ইত্যাদি), আর React / Next.js / Vue দিয়ে সেই ডাটা ওয়েবসাইটে দেখাবেন।

Real Life উদাহরণ

ধরুন আপনি একটা ই-কমার্স ওয়েবসাইট বানাচ্ছেন — যেমন PickHub নামের একটা অনলাইন শপ।

- Strapi = দোকানের গুদাম ঘর (warehouse) + হিসাবরক্ষক। এখানে প্রোডাক্টের নাম, দাম, ছবি, স্টক — সবকিছু সংরক্ষিত থাকে।
- React Frontend = দোকানের shop front, যেখানে কাস্টমার এসে প্রোডাক্ট দেখে এবং কেনে।

 **মনে রাখুন:** দোকানের মালিক (আপনি) গুদাম ঘরে (Strapi Admin Panel) গিয়ে নতুন প্রোডাক্ট যোগ করেন বা দাম পরিবর্তন করেন — আর দোকানের সামনের অংশ (React) সাথে সাথে সেই আপডেট দেখাতে শুরু করে, কোনো কোড পরিবর্তন ছাড়াই।

কেন Strapi ব্যবহার করব?

সুবিধা	ব্যাখ্যা
 কোড ছাড়াই ডাটাবেস	Content-Type Builder দিয়ে drag-and-drop স্টাইলে ফিল্ড বানানো যায়
 Auto REST / GraphQL API	প্রতিটা Content-Type বানাতেই স্বয়ংক্রিয়ভাবে API তৈরি হয়
 Role-based Access	কে কোন ডাটা দেখতে/পরিবর্তন করতে পারবে তা নিয়ন্ত্রণ করা যায়
 Self-hosted অথবা Cloud	নিজের সার্ভারে অথবা Strapi Cloud-এ হোস্ট করা যায়
 ওপেন সোর্স	সম্পূর্ণ বিনামূল্যে ব্যবহারযোগ্য

২. প্রয়োজনীয় জিনিসপত্র (Prerequisites)

প্রজেক্ট শুরুর আগে আপনার কম্পিউটারে এগুলো থাকতে হবে:

- Node.js (LTS ভার্সন) — nodejs.org থেকে ডাউনলোড করুন
- npm (Node.js এর সাথে স্বয়ংক্রিয়ভাবে ইনস্টল হয়)
- VS Code (অথবা যেকোনো কোড এডিটর)
- একটি স্থিতিশীল ইন্টারনেট কানেকশন

ইনস্টল ঠিকঠাক হয়েছে কিনা যাচাই করতে টার্মিনালে লিখুন:

```
node -v  
npm -v
```

ভার্সন নাম্বার দেখালে বুঝবেন ইনস্টলেশন সঠিকভাবে হয়েছে।

৩. Strapi প্রজেক্ট তৈরি করা

ধাপ ১ — Strapi ওয়েবসাইটে যাওয়া

strapi.io ওয়েবসাইটে গিয়ে "Get Started" বাটনে ক্লিক করুন। সেখান থেকে ইনস্টলেশন কমান্ড (CLI command) কপি করুন।

ধাপ ২ — টার্মিনালে কমান্ড রান করা

```
npx create-strapi-app@latest my-project
```

এই কমান্ড রান করলে কিছু প্রশ্ন আসবে:

1. Project name? → আপনার প্রজেক্টের নাম দিন
2. Use TypeScript? → চাইলে Yes/No দিন (নতুনদের জন্য No সহজ)
3. Default database (SQLite)? → Yes দিন (লোকাল ডেভেলপমেন্টের জন্য সবচেয়ে সহজ)
4. Start with example data? → আপনার ইচ্ছা অনুযায়ী বেছে নিন

 **টিপস:** এরপর VS Code-এ গিয়ে প্রজেক্ট ফোল্ডারটি ওপেন করুন। টার্মিনাল থেকে `code .` কমান্ড দিয়েও সরাসরি খোলা যায়।

৪. প্রজেক্ট রান করা

Strapi-তে তিনটা গুরুত্বপূর্ণ কমান্ড আছে — এগুলোর পার্থক্য বোঝা খুবই জরুরি:

কমান্ড	কখন ব্যবহার	কী করে
npm run develop	Local development-এর সময়	Watch Mode-এ চলে — কোড পরিবর্তন সাথে সাথে রিফ্রেক্ট হয়, Content-Type Builder এডিট করা যায়
npm run build	Production-এর আগে	Admin Panel-কে স্ট্যাটিক ফাইলে কম্পাইল করে
npm run start	Production mode-এ	Watch Mode ছাড়াই দ্রুতগতিতে রান করে

```
# ডেভেলপমেন্টের সময়
npm run develop

# Production-এ যাওয়ার আগে
npm run build
npm run start
```

 **মনে রাখুন:** "Develop" মানে আপনি বানাচ্ছেন/পরিবর্তন করছেন। "Build + Start" মানে বানানো শেষ, এখন শুধু চালাচ্ছেন — অনেকটা বাড়ি বানানো শেষে শুধু থাকা শুরু করার মতো।

npm run develop রান করার পর ব্রাউজারে যান:

```
http://localhost:1337/admin
```

৫. Admin Panel — সাইনআপ ও Dashboard পরিচিতি

প্রথমবার `http://localhost:1337/admin` -এ গেলে একটা Signup ফর্ম আসবে। এখানে নাম, ইমেইল, পাসওয়ার্ড দিয়ে Admin অ্যাকাউন্ট তৈরি করুন (এটা সুপার-অ্যাডমিন অ্যাকাউন্ট, কাস্টমার অ্যাকাউন্ট নয়)।

সাইনআপের পর Dashboard-এ পৌঁছাবেন। বাম পাশের মেনুতে যা যা দেখবেন:

-  Home — সম্পূর্ণ ওভারভিউ
-  Content Manager — ডাটা এন্ট্রি/এডিট করার জায়গা
-  Media Library — ফাইল/ছবি আপলোড করার জায়গা
-  Content-Type Builder — ডাটাবেস স্ট্রাকচার ডিজাইন করার জায়গা
-  Plugins — এক্সট্রা ফিচার যোগ করার জায়গা
-  Settings — সব সেটিংস (Roles, API Tokens, i18n ইত্যাদি)

৬. Content-Type Builder

এখানেই আপনি ঠিক করেন আপনার ডাটাবেসে কী কী টেবিল থাকবে এবং প্রতিটোতে কী কী ফিল্ড থাকবে।

দুই ধরনের Content-Type

টাইপ	কখন ব্যবহার	উদাহরণ
Collection Type	একই ধরনের একাধিক এন্ট্রি থাকলে	Products, Blog Posts, Users
Single Type	একটাই এন্ট্রি থাকলে	About Us পেজ, Homepage Settings

ধাপে ধাপে নতুন Content-Type বানানো (উদাহরণ: "Product")

- Content-Type Builder → Create new collection type ক্লিক করুন
- Display name দিন (যেমন: Product) → Continue চাপুন
- Add New Field-এ ক্লিক করে ফিল্ড টাইপ বাছাই করুন

ফিল্ড	টাইপ	উদাহরণ
Title/Name	Text	"CyberBoard Neo Mechanical Keyboard"
Price	Number	189
Description	Rich Text / Long Text	প্রোডাক্টের বিস্তারিত বর্ণনা
Category	Text বা Relation	"Keyboards"
Image	Media	প্রোডাক্টের ছবি
Rating	Number	4.9
In Stock	Boolean	true / false
Features	JSON বা Repeatable Component	["Gasket Mounted", "Hot-Swappable"]

- প্রতিটা ফিল্ড যোগ করার পর Finish চাপুন
- একাধিক ফিল্ড যোগ করতে Add another field চাপুন
- সব ফিল্ড যোগ হয়ে গেলে উপরে ডান দিকে Save চাপুন

⚠ সতর্কতা: Save করলে Strapi সার্ভার স্বয়ংক্রিয়ভাবে রিস্টার্ট হবে — এটা সম্পূর্ণ স্বাভাবিক আচরণ, ভয় পাওয়ার কিছু নেই।

৭. Content Manager — CRUD অপারেশন

Content-Type বানানো শেষে, এখন এখানে আসল ডাটা ভরবেন। CRUD মানে:

সংক্ষেপ	পূর্ণরূপ	অর্থ
C	Create	তৈরি করা
R	Read	পড়া / দেখা
U	Update	পরিবর্তন করা
D	Delete	মুছে ফেলা

Strapi Admin Panel-এ CRUD

৪. বাম মেনু থেকে Content Manager → আপনার বানানো Content-Type (যেমন Product) সিলেক্ট করুন
৯. Create new entry ক্লিক করে নতুন প্রোডাক্ট যোগ করুন (Create)
১০. লিস্টে যেকোনো এন্ট্রিতে ক্লিক করলে তার ডিটেইল দেখা যাবে (Read)
১১. এডিট করে Save করলে আপডেট হবে (Update)
১২. এন্ট্রি সিলেক্ট করে ডিলিট বাটনে ক্লিক করলে মুছে যাবে (Delete)
১৩. সবশেষে Publish বাটনে ক্লিক করতে ভুলবেন না

⚠ সতর্কতা: এন্ট্রি Draft অবস্থায় থাকলে সেটা পাবলিক API-তে দেখা যাবে না — অবশ্যই Publish করতে হবে।

কোড দিয়ে CRUD (React থেকে API কল)

আপনার React অ্যাপ থেকে একই কাজ API কল দিয়ে করা যায়:

CREATE — নতুন আইটেম যোগ করা:

```
const res = await fetch(`${API_URL}/wishlist-items`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    data: { guestId, product: product.id }
  })
});
```

READ — সব ডাটা আনা:

```
const res = await fetch(`${API_URL}/products?populate=*`);  
const json = await res.json();
```

DELETE — আইটেম মুছে ফেলা:

```
const res = await fetch(`${API_URL}/wishlist-items/${itemId}`, {  
  method: "DELETE"  
});
```

 **মনে রাখুন:** populate=* মানে হলো — শুধু মূল ডাটা না, এর সাথে রিলেটেড সব ডাটাও (যেমন ছবি, ক্যাটাগরি) একসাথে আনো।

৮. Media Library

এটা আপনার সব ফাইল, ছবি, ভিডিও রাখার জায়গা — অনেকটা গুগল ড্রাইভের মতো।

- বাম মেনু থেকে Media Library-তে যান
- Add new assets বাটনে ক্লিক করে কম্পিউটার থেকে ফাইল আপলোড করুন
- এই আপলোড করা ফাইলগুলো পরে Content-Type এর "Media" ফিল্ডে ব্যবহার করা যাবে (যেমন প্রোডাক্ট ছবি)

React-এ ছবির URL ব্যবহার করা

Strapi থেকে আসা ছবির URL সাধারণত একটা relative path হিসেবে আসে (যেমন /uploads/photo.jpg)। এটাকে পুরো URL বানাতে হবে:

```
const API_URL = import.meta.env.VITE_STRAPI_API_URL; // http://localhost:1337/api
const BASE_URL = API_URL.replace("/api", ""); // http://localhost:1337

const imageUrl = item.image ? `${BASE_URL}${item.image.url}` : "";
```

৯. Roles & Permissions

এখানেই আপনি ঠিক করেন কে কোন API ব্যবহার করতে পারবে এবং কী করতে পারবে।

Real Life উদাহরণ

ধরুন আপনার দোকানে দুই ধরনের মানুষ আসে:

-  সাধারণ ভিজিটর (Public) — শুধু প্রোডাক্ট দেখতে পারবে, কিনতে/এডিট করতে পারবে না
-  লগইন করা কাস্টমার (Authenticated) — প্রোডাক্ট দেখতে পারবে, উইশলিস্টে যোগ করতে পারবে, অর্ডার করতে পারবে

সেটাপ করার ধাপ

14. Settings → Users & Permissions Plugin → Roles -এ যান
15. Public রোলে ক্লিক করুন — কোন Content-Type এর কোন কোন অ্যাকশন (find, findOne, create, update, delete) পাবলিক ইউজার করতে পারবে তা ঠিক করুন
16. Authenticated রোলে গিয়ে একই কাজ করুন — প্রয়োজনে create, update ইত্যাদিও দিতে পারেন
17. পরিবর্তন করার পর Save চাপুন

 **সতর্কতা:** Permission না দিলে আপনার API কল 403 Forbidden এরর দেবে। অনেক নতুন ডেভেলপার এই ভুলটা করেন — কোড ঠিক থাকা সত্ত্বেও Permission সেট না করায় API কাজ করে না।

১০. API Tokens

কখনো কখনো আপনি চাইবেন সার্ভার-টু-সার্ভার কমিউনিকেশনের জন্য একটা নিরাপদ key ব্যবহার করতে (Public/Authenticated role ছাড়াই)।

18. Settings → API Tokens → Create new API Token

19. নাম দিন, Token duration (Unlimited বা নির্দিষ্ট সময়) সিলেক্ট করুন

20. Token type বাছাই করুন: Read-only / Full access / Custom

21. Save করার সাথে সাথেই টোকেনটি একবার দেখানো হবে — এটা কপি করে নিরাপদ জায়গায় রাখুন

⚠ সতর্কতা: টোকেনটি একবার দেখানোর পর আর দেখা যাবে না — এই মুহূর্তেই কপি করে রাখতে ভুলবেন না।

ব্যবহার করার সময় হেডারে পাঠাতে হয়:

```
fetch(`${API_URL}/products`, {  
  headers: {  
    Authorization: `Bearer YOUR_API_TOKEN_HERE`  
  }  
});
```

১১. Internationalization (i18n)

যদি আপনার ওয়েবসাইট একাধিক ভাষায় থাকে (যেমন বাংলা + ইংরেজি), তাহলে এই প্লাগিন কাজে লাগে।

22. Settings → Internationalization

23. Add new locale ক্লিক করুন

24. Basic Settings-এ ভাষা সিলেক্ট করুন (Bengali, English ইত্যাদি)

25. Advanced Settings-এ গিয়ে "Set as default locale" এ TRUE/FALSE সিলেক্ট করুন

26. Save করুন

 **মনে রাখুন:** একটা ভাষা সবসময় default হিসেবে থাকতেই হবে। এরপর প্রতিটা Content-Type-এ গিয়ে i18n এনাবল করলে একই প্রোডাক্ট দুই ভাষাতেই লেখা যাবে।

১২. Environment Variables (.env) সেটআপ

প্রজেক্ট ফোল্ডারের একদম উপরে .env নামের একটা ফাইল থাকে — এখানে গুরুত্বপূর্ণ ও গোপনীয় তথ্য (API URL, পাসওয়ার্ড) রাখা হয়, যাতে কোডের মধ্যে সরাসরি না লিখতে হয়।

React (Frontend) সাইডে — .env

```
VITE_STRAPI_API_URL=http://localhost:1337/api
```

কোডে ব্যবহার (Vite প্রজেক্টে)

```
const API_URL = import.meta.env.VITE_STRAPI_API_URL;
```

 **মনে রাখুন:** Local-এ ডেভেলপ করার সময় URL হবে localhost:1337, কিন্তু Production-এ আসল ডোমেইন URL হবে। .env ফাইলের শুধু একটা লাইন পরিবর্তন করলেই পুরো প্রজেক্ট নতুন URL-এ কানেক্ট হয়ে যাবে — কোড পরিবর্তন করতে হবে না।

১৩. React ফ্রন্টএন্ডের সাথে Strapi কানেক্ট করা

এখন আমরা একটা সম্পূর্ণ প্র্যাকটিক্যাল উদাহরণ দেখব — কীভাবে React Context API ব্যবহার করে পুরো প্রজেক্টে Strapi-র ডাটা শেয়ার করা যায়।

ধাপ ১ — Context ফাইল তৈরি করা (StrapiContext.jsx)

```
import React, { createContext, useContext, useEffect, useState } from "react";
import { mockProducts } from "../mockData";

const API_URL = import.meta.env.VITE_STRAPI_API_URL;
const BASE_URL = API_URL.replace("/api", "");

const StrapiContext = createContext(null);

export const StrapiProvider = ({ children }) => {
  const [products, setProducts] = useState([]);
  const [wishlist, setWishlist] = useState([]);
  const [loading, setLoading] = useState(false);
```

ব্যাখ্যা:

- `createContext(null)` — একটা "শেয়ার্ড বক্স" তৈরি করছি যেখানে ডাটা রাখা যাবে এবং পুরো অ্যাপের যেকোনো কম্পোনেন্ট থেকে অ্যাক্সেস করা যাবে।
- `mockProducts` — Strapi থেকে ডাটা আসার আগে, বা ইন্টারনেট না থাকলে, ব্যাকআপ/ডেমো ডাটা হিসেবে ব্যবহারের জন্য রাখা হয়েছে।

ধাপ ২ — Guest ID তৈরি করা (যারা লগইন করেনি তাদের জন্য)

```
const [guestId] = useState(() => {
  let id = localStorage.getItem("pickhub_guest_id");
  if (!id) {
    id = `guest_${Math.random().toString(36).substring(2, 11)}`;
    localStorage.setItem("pickhub_guest_id", id);
  }
  return id;
});
```



মনে রাখুন: এটা অনেকটা দোকানে ঢুকলে একটা টোকেন নাম্বার দেওয়ার মতো — যাতে কাস্টমার লগইন না করেও তার উইশলিস্ট মনে রাখা যায় (browser-এর localStorage ব্যবহার করে)।

ধাপ ৩ — Strapi থেকে প্রোডাক্ট আনা


```
const getProducts = async () => {
  setLoading(true);
  try {
    const res = await fetch(`${API_URL}/products?populate=*`);
    const json = await res.json();

    const mapped = json.data.map(item => ({
      id: item.documentId,
      name: item.name,
      price: item.price,
      description: item.description,
      category: item.category,
      image: item.image ? `${BASE_URL}${item.image.url}` : "",
      rating: item.rating,
      inStock: item.inStock,
      features: item.features || [],
    }));

    setProducts(mapped);
  } catch (error) {
    console.error("Error fetching products:", error);
    setProducts(mockProducts); // ব্যর্থ হলে ডেমো ডাটা দেখাও
  } finally {
    setLoading(false);
  }
};
```

কোড	কাজ
setLoading(true)	লোডিং স্পিনার দেখানোর জন্য ক্ল্যাগ চালু করা
fetch(...)	Strapi API-তে রিকোয়েস্ট পাঠানো
json.data.map(...)	জটিল ডাটা স্ট্রাকচারকে সহজ ফরম্যাটে রূপান্তর করা
item.documentId	Strapi v5-এ প্রতিটা এন্ট্রির ইউনিক আইডি
catch (error)	ইন্টারনেট/সার্ভার সমস্যায় অ্যাপ যেন না ভাঙে, তাই mock ডাটা ফলব্যাক
finally	সফল হোক বা ব্যর্থ — শেষে লোডিং বন্ধ করা আবশ্যিক

১৪. Real Project Example — Wishlist ফিচার

এটা একটা সম্পূর্ণ CRUD-এর বাস্তব উদাহরণ — প্রোডাক্ট উইশলিস্টে যোগ করা ও মুছে ফেলা।

Add to Wishlist (Create)

```
const addToWishlist = async (product) => {
  try {
    const res = await fetch(`${API_URL}/wishlist-items`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        data: { guestId, product: product.id }
      })
    });
  } catch (error) {
    console.error("Error adding to wishlist:", error);
  }
};
```

Remove from Wishlist (Delete)

```
const removeFromWishlist = async (productId) => {
  const item = wishlist.find(i => i.id === productId);
  try {
    const res = await fetch(`${API_URL}/wishlist-items/${item.wishlistItemId}`, {
      method: "DELETE"
    });
    if (res.ok) {
      setWishlist(prev => prev.filter(i => i.id !== productId));
    }
  } catch (error) {
    console.error("Error removing from wishlist:", error);
  }
};
```

ব্যাখ্যা:

- প্রথমে wishlist অ্যারে থেকে খুঁজে বের করছি কোন আইটেমটা মুছতে হবে
- Strapi-কে DELETE রিকোয়েস্ট পাঠাচ্ছি সেই আইটেমের আইডি দিয়ে
- সফল রেসপন্স (res.ok) পেলে লোকাল state থেকেও আইটেমটি সরিয়ে ফেলছি — যাতে UI সাথে সাথে আপডেট হয়, পেজ রিফ্রেশ ছাড়াই



মনে রাখুন: এই প্যাটার্নটা মনে রাখুন: Backend-এ ডাটা পরিবর্তন করো → সফল হলে Frontend-এর state-ও পরিবর্তন করো। এটাই প্রায় সব CRUD অপারেশনের মূল নিয়ম।

১৫. Deployment — Hostinger VPS + Coolify

ডেভেলপমেন্ট শেষে, এখন প্রজেক্টকে ইন্টারনেটে লাইভ করার পালা। এখানে আমরা Self-hosted পথ বেছে নিচ্ছি (Strapi Cloud-এর বদলে), কারণ এতে Strapi-কে একটা production system হিসেবে শেখা যায় এবং খরচও কম হয়।

Deployment-এর সম্পূর্ণ ফ্লো

Local Setup → Hostinger VPS → Dashboard → Permissions →
React Frontend → Real API → Live Backend

ধাপে ধাপে

27. Hostinger VPS Plan কেনা — একটা VPS (Virtual Private Server) সাবস্ক্রাইব করুন
28. Coolify ইনস্টল ও Deploy — Coolify একটা টুল যেটা VPS-এ অ্যাপ ডিপ্লয় করা সহজ করে দেয়। প্রজেক্ট কানেক্ট করে Deploy ক্লিক করুন, শেষে একটা লিংক পাবেন
29. লিংকে ক্লিক করে লাইভ Strapi Dashboard-এ যান
30. Dashboard Base URL কপি করে React প্রজেক্টের .env ফাইলে পুরনো localhost URL রিপ্লেস করুন
31. Production-এও Roles & Permissions আবার চেক করুন (Local-এর সেটিংস স্বয়ংক্রিয়ভাবে VPS-এ কপি হয় না)
32. Production-এ build + start কমান্ড রান হবে (Coolify সাধারণত এটা স্বয়ংক্রিয়ভাবে করে দেয়)

.env ফাইলের পরিবর্তন:

```
# আগে (Local)
VITE_STRAPI_API_URL=http://localhost:1337/api

# পরে (Production)
VITE_STRAPI_API_URL=https://your-live-strapi-url.com/api
```

⚠ সতর্কতা: Production server-এ কখনো npm run develop চালাবেন না — এটা শুধু Local development-এর জন্য। Production-এ সবসময় build + start ব্যবহার করুন, এতে সার্ভার দ্রুত ও নিরাপদ থাকে।

১৬. সাধারণ সমস্যা ও সমাধান

সমস্যা	কারণ	সমাধান
API কল করলে 403 Forbidden	Roles & Permissions-এ অ্যাকশন এনাবল করা নেই	Settings → Roles → চেকবক্স টিক দিন → Save
Admin Panel-এ ডাটা আছে, কিন্তু API-তে নেই	এন্ট্রিটি এখনো Draft অবস্থায়	Content Manager-এ গিয়ে Publish চাপুন
ছবি দেখাচ্ছে না (broken image)	শুধু relative path ব্যবহার হচ্ছে	BASE_URL + item.image.url দিয়ে ফুল URL বানান
.env পরিবর্তন করেও কাজ করছে না	Dev server পুরনো .env মনে রেখে দিয়েছে	সার্ভার বন্ধ করে আবার npm run develop চালান
Content-Type Builder এডিট করা যাচ্ছে না	আপনি npm run start mode-এ আছেন	npm run develop mode-এ যান
Save করার পর সার্ভার রিস্টার্ট হচ্ছে	এটা স্বাভাবিক আচরণ	কিছু সেকেন্ড অপেক্ষা করুন

১৭. Quick Cheat Sheet

কমান্ডসমূহ

<code>npx create-strapi-app@latest my-project</code>	# নতুন প্রজেক্ট
<code>npm run develop</code>	# ডেভেলপমেন্ট মোড
<code>npm run build</code>	# প্রোডাকশনের জন্য বিল্ড
<code>npm run start</code>	# প্রোডাকশন মোড

গুরুত্বপূর্ণ URL

<code>http://localhost:1337/admin</code>	# Admin Panel
<code>http://localhost:1337/api</code>	# API base
<code>http://localhost:1337/documentation</code>	# API Docs

CRUD-এর জন্য Fetch প্যাটার্ন

```
// GET (Read)
fetch(`${API_URL}/products?populate=*`)

// POST (Create)
fetch(`${API_URL}/products`, { method: "POST", headers: {...}, body: JSON.stringify({
data: {...} }) })

// PUT (Update)
fetch(`${API_URL}/products/${id}`, { method: "PUT", headers: {...}, body:
JSON.stringify({ data: {...} }) })

// DELETE
fetch(`${API_URL}/products/${id}`, { method: "DELETE" })
```

কাজের ক্রম (Workflow Order)

1. Strapi প্রজেক্ট তৈরি
2. `npm run develop`
3. Admin Signup
4. Content-Type Builder দিয়ে স্ট্রাকচার বানানো
5. Content Manager দিয়ে ডাটা ভরা ও Publish করা
6. Roles & Permissions সেট করা
7. `.env`-এ API URL বসানো
8. React থেকে fetch করে ডাটা আনা

9. CRUD ফিচার বানানো (Add/Remove ইত্যাদি)
10. `npm run build` → `npm run start` (টেস্ট)
11. VPS + Coolify দিয়ে Deploy
12. Production .env আপডেট করা
13. Production Roles & Permissions আবার চেক করা

শেষ কথা

Strapi শেখার মূল চাবিকাঠি হলো এটাকে একটা গুদাম ঘর + হিসাবরক্ষক হিসেবে চিন্তা করা, যেটা আপনার ফ্রন্টএন্ডকে (React/Vue/Next.js) ডাটা সরবরাহ করে API-র মাধ্যমে। একবার এই মানসিক চিত্রটা পরিষ্কার হয়ে গেলে, Content-Type Builder, Permissions, Deployment — সবকিছুই যৌক্তিকভাবে সহজ মনে হবে।

 **শেষ পরামর্শ:** যদি কখনো আটকে যান, প্রথমে চেক করুন: (১) Permission ঠিক আছে কি না, (২) এন্ট্রি Publish করা আছে কি না, (৩) .env-এর URL সঠিক কি না। ৯০% সমস্যা এই তিনটার একটায় হয়।